

chap: Sports Store: A Real Application

Creating the Sports Store Project

```
dotnet new globaljson --sdk-version 6.0.100 --output SportsSln/SportsStore
dotnet new web --no-https --output SportsSln/SportsStore
--framework net 6.0
dotnet new sln -o SportsSln
dotnet sln SportsSln add SportsSln/SportsStore
```

Creating the Unit Test Project:

```
dotnet new xunit -o SportsSln/sportsStore.Tests --framework net 6.0
dotnet sln SportsSln add SportsSln/SportsStore.Tests
dotnet add SportsSln/SportsStore.Tests reference SportsSln/SportsStore
```

Installing the Moq Package:

```
dotnet add SportsSln/SportsStore.Tests package Moq --Version 4.16.1
```

Opening the Project:

Configuring the HTTP port:

- Setting the HTTP port in the launchSettings.json file in the SportsStore/Properties folder,

```
{
  "IISSettings": {
```

Date _____
Page _____

```

"WindowsAuthentication": false,
"anonymousAuthentication": true,
"iisExpress": {
  "applicationUrl": "http://localhost:5000",
  "sslPort": 0
},
"profiles": {
  "SportsStore": {
    "commandName": "Project",
    "dotnetRunMessages": true,
    "launchBrowser": true,
    "applicationUrl": "http://localhost:5000",
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  },
  "IIS EXPRESS": {
    "commandName": "IISExpress",
    "launchBrowser": true,
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  }
}
}
}

```

Creating the Application Project folders:

- The Application Project folders:

- | Name | Description |
|------------------|--|
| (1) Models | This folder will contain the data model and the classes that provide access to the data in the application's database. |
| (2) Controllers | This folder will contain the controller classes that handle HTTP requests. |
| (3) Views | This folder will contain all the PAGES files, grouped into separate subfolders. |
| (4) Views/Home | This folder will contain PAGES files that are specific to the Home Controller, which I create in the "Creating the controllers and views" section. |
| (5) Views/Shared | This folder will contain PAGES files that are common to all controllers. |

Preparing the Application Services and Request Pipeline:

- Configuration the application in the Program.cs file in the SportsStore folder,

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
// app.MapGet("/", () => "Hello World!");
app.UseStaticFiles();
app.MapDefaultControllerRoute();
app.Run();
```

The `builder.Services` property is used to set up objects, known as `Separate Services`, that can be used throughout the application and that can be accessed through a feature called `dependency injection`, which I describe in chap 14.

The `AddControllers` with `Views` method sets up the shared objects required by applications using the MVC framework and the `Razor` view engine.

ASP.NET Core receives HTTP requests and passes them along a request pipeline, which is populated with `Middleware` components registered using the `app` property. Each `middleware` component is able to produce the requests, modify them, generate a response, or modify the responses that other components have produced.

The request pipeline is the heart of ASP.NET Core, and I describe it in details in chap 12, where I also explain how to create custom `middleware` components.

The contents of the `_Layout.cshtml` file in the `SportsStore/Views/Shared` folder,

```
<!DOCTYPE html>
<html>
<head>
  <meta name = "viewport" content = "width =
    device-width" />
  <title> SportsStore </title>
</head>
<body>
  <div> @RenderBody() </div>
</body>
</html>
```

Starting data model:

- The contents of the Product.cs file in the SportsStore / Models folder,

```
using System.ComponentModel.DataAnnotations.Schema;
namespace SportsStore.Models {
```

```
    public class Product {
        public long? ProductID { get; set; }
```

```
        public string Name { get; set; } = String.Empty;
```

```
        public string Description { get; set; } =
            String.Empty;
```

```
        [Column(TypeName = "decimal(8,2)")]
```

```
        public decimal Price { get; set; }
```

```
        public string Category { get; set; } = String.Empty;
```

```
    }
```

Adding Data to the Application:

Now that the SportsStore contains some basic setup and can produce a simple response, it is time to add some data so that the application has something more useful to display. The SportsStore application will store its data in a SQL Server LocalDB database, which is accessed using Entity Framework Core. Entity Framework Core is the Microsoft Object-to-relational mapping (ORM) framework, and it is the most widely used method of accessing databases in ASP.NET Core projects.

Installing the Entity Framework Core Packages:

- Adding the Entity Framework Core Packages to the SportsStore project,

```
dotnet add package Microsoft.EntityFrameworkCore.  
Design -- Version 6.0.0
```

```
dotnet add package Microsoft.EntityFrameworkCore.  
SqlServer -- Version 6.0.0
```

- Adding Installing the Entity Framework Core Tool Package:

```
dotnet tool uninstall --global dotnet-ef  
dotnet tool install --global dotnet-ef  
-- Version 6.0.0
```

Defining the Connecting String:

- Adding a configuration Setting in the appsettings.json file in the SportsStore folder,

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*",  
  "ConnectionStrings": {  
    "SportsStoreConnection": "Server=(localdb)\\MSSQL  
LocalDB; Database = SportsStore;"
```

MultipleActiveResultSets = true"

```
}  
}
```

Creating the Database Context Class:

- The Contents of the StoreDbContext.cs file in the SportsStore / Models folder,

.. using Microsoft.EntityFrameworkCore;

```
namespace SportsStore.Models {  
    public class StoreDbContext : DbContext {  
        public StoreDbContext(DbContextOptions  
            <StoreDbContext> options)  
            : base(options) {}  
        public DbSet<Product> Products =>  
            Set<Product>();  
    }  
}
```

A Real ASP.NET Core Application: Theory Notes Creating a Project.

(1) Adding Data to the Application:

- In any application, data is central to how information is handled and displayed.
- To create a data-driven application, follows these steps:

- Define Models:
- A model is a C# class that represents the data structure of an object in the Systems.
- Example: Product model with properties like Name, Description, Price and category.
- Add a database Context.
- Use Entity framework Core (EF Core) to interact with the database.
- Create ApplicationDbContext Inheriting from DbContext with DbSet<T> properties mapping to databases tables.
- Data Seeding: Seed initial data so the application has sample products.

(2) Displaying a List of Products:

- After adding data, display it to the user.

(i) Controllers: Create product controller to retrieve products from the database.

- Example:

```
public IActionResult Index() {  
    var products = - Context.products.ToList();  
    return View(products);  
}
```

(ii) Views: Use Razor Syntax to loop through products and render them.

- Example:

```
@foreach (var product in model) {  
    <div>  
        <h2>@product.Name </h2>  
        <p>@product.Description </p>  
    }
```

```

    <span>₹@productPrice </span>
  </div>
}

```

(iii) Adding Pagination:

- Pagination improves Performance and UX by Splitting data into pages:
- Use LINQ:


```

int pageSize = 10;
var products = - context.Products.Skip((pageNumber-1)
                                * pageSize)
                                .Take(pageSize)
                                .ToList();

```
- Provide UI links for next and previous.

(iv) Styling the context:

- Use Bootstrap for a responsive, modern look.
- Arrange Product in grids or Cards.
- Example,

```

<div class = "Card">
  <div class = "Card-body">
    <h5>@product.Name </h5>
    <p>@product.Description </p>
    <span> ₹ @product.price </span>
  </div>
</div>

```

chap: SportsStore: Navigation and Cart

Date _____
Page _____

Adding Navigation Controls:

- Build a header/navbar in layout.cshtml:

```
<nav>
  <ul>
    <li><a href="@Url.Action("Index", "Home")">
      Home </a> </li>
    <li><a href="@Url.Action("Index", "Products")">
      Products </a> </li>
    <li><a href="@Url.Action("Index", "Cart")">
      Cart </a> </li>
  </ul>
</nav>
```

- Ensure, it is visible on all pages via the layout.

Building the Shopping Cart:

- Cart Models:

```
public class CartItem {
    public Product product {get; set;}
    public int Quantity {get; set;}
}
```

- Sessions:

Store cart data so it persists across pages.

```
var cart = HttpContext.Session.GetObject<List<CartItem>>("Cart") ??
```

```
new List<CartItem>();
```

```
Cart.Add(new CartItem ( product = product,
                        Quantity = 1);
```

```
HttpContext.Session.SetObject("cart", cart); }
```

Completing the cart:

- Refining the cart Model With a Services:

Encapsulate logic in a Service (Single Responsibility Principle):

```
public class CartServices : ICartService {
    private readonly IHttpContextAccessor contextAccessor;
    public CartServices (HttpContextAccessor accessor) {
        _contextAccessor = accessor;
    }
    public void AddToCart (product product) {
        var cart = GetCart();
        var item = cart.FirstOrDefault(c => c.product.
            Id == product.Id);
        if (item == null) {
            cart.Add (new CartItem { product = product,
                Quantity = 1 });
        }
        else {
            item.Quantity++;
        }
    }
    public List<CartItem> GetCart () {
        return
            contextAccessor.HttpContext.Session.GetObject
            <List<CartItem>> ("cart")
            ?? new List<CartItem>();
    }
}
```

- Completing the cart functionality:

- Create a CartController to handle View Cart, Remove Items, and Update Quantities.
- Display Cart Summary showing items, quantities, and total.

- Submitting Orders:

- Order Model:

```
public class Order {  
    public int id {get; set;}  
    public String CustomerName {get; set;}  
    public List<CartItem> Items {get; set;}  
    public DateTime OrderDate {get; set;}  
    public decimal Total {get; set;}  
}
```

- Save order in the database on checkout and clear the cart.

Role - Based authorization in ASP.NET Core:

- Add Roles Services to Identity:

Register role-based authorization services in Program.cs by calling AddRoles with the role type in the app's identity configuration. The role type in the following example is IdentityRole:

```
builder.Services.AddDefaultIdentity<IdentityUser>(...)  
    .AddRoles<IdentityRole>()  
    ...
```

The preceding code requires the Microsoft.AspNetCore.Identity.UI package and a using directive for Microsoft.AspNetCore.Identity.

Adding role checks:

- Role-based authorization checks:

- Are declarative and specify roles which the current user must be a member of to access the requested resource.
- Are applied to Razor Pages, Controllers or actions within a controller.
- Can not be applied at the Razor pages handler level, they must be applied to the page.

- For example,

Single role can be specified by,

```
[Authorize(Roles = "Administrator")]  
public class AdministrationController : Controller {  
    public IActionResult Index() =>  
        Context("Administrator"); } }
```

- Example, Multiple role can be specified as a comma separated list:

```
[Authorize(Roles = "HRManager.finance")]  
public class SalaryController : Controller {  
    public IActionResult Payslip() =>  
        Context("HRManager || finance"); } }
```

- for example,

```
[Authorize (Roles = "PowerUser")]  
[Authorize (Roles = "ControlPanelUser")]  
public class ControlPanelController : Controller {  
    public IActionResult Index() =>  
        Content("PowerUser && ControlPanelUser");  
}
```

- In the preceding, "ControlAllPanelController" Controller

- Members of the Administrator role or the powerUser role can access the controller and the setTime action.
- Only members of the Administrator role can access the shutdown action.

Example,

```
[Authorize (Roles = "Administrator, PowerUser")]  
public class ControlAllPanelController : Controller  
{  
    public IActionResult SetTime() =>  
        Content("Administrator || PowerUser");  
    [Authorize (Roles = "Administrator")]  
    public IActionResult Shutdown() =>  
        Content("Administrator only");  
}
```

- A Controller can be secured but allow anonymous, Unauthenticated access to individual actions:

[Authorize]

```
public class Control3PanelController : Controller {
    public IActionResult SetTime() =>
        Content("[Authorize]");
    [AllowAnonymous]
    public IActionResult Login() =>
        Content("[AllowAnonymous]");
}
```

Policy Based role checks:

- Role requirements can also be expressed using the policy syntax, when a developer registers a policy at application startups as part of the Authorization service configuration. This typically occurs in the Program.cs file:-

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages();
builder.Services.AddControllersWithViews();
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("Require AdministratorRole",
        policy => policy.RequireRole("Administrator"));
});
var app = builder.Build();
if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}
```

Custom authorization Policies With IAuthorizationRequirementData :

(1) Minimum age authorize attributes:

- The "MinimumAgeAuthorizeAttribute" implementation of `IAuthorizationRequirementData` Sets an authorization age:

```

Using Microsoft.AspNetCore.Authorization;
namespace AuthRequirementData.Authorization;
class MinimumAgeAuthorizeAttribute (int age):
    AuthorizeAttribute, IAuthorizationRequirement,
    IAuthorizationRequirementData
{
    public int Age { get; set; } = age;
    public IEnumerable<IAuthorizationRequirement>
        GetRequirements () {
            yield return this; }
}
    
```

(2) Minimum age authorization handler:

- The "MinimumAgeAuthorizationHandler" class handles the single `IAuthorizationRequirement` provided by "MinimumAgeAuthorizeAttribute", as specified by the generics parameter "MinimumAgeAuthorizeAttribute".
- The "HandleRequirementAsync" Method:
 - Gets the User's birth data claim.
 - Obtains the User's age from the claim.
 - Adjusts age if the user hasn't had a birthday this year.
 - Marks the authorization requirement Succeeds if the User meets the age requirements.

- Implements logging for demonstration purposes.

Get Started with the data protection APIs in ASP.NET Core :-

- Basically, protecting data consists of the following steps:

- (1) Create a data protector from a data protection provider.
- (2) Call the "protect" method with the data you want to protect.
- (3) Call the "Unprotect" method with the data you want to turn back into plain text.

- Most frameworks and app models, such as ASP.NET Core or SignalR, already configure the data protection system and add it to a services container that is accessed via "dependency injection". The following sample demonstrate:

- (a) Configuration a service container for dependency injection and registering the data protection stack.
- (b) Receiving the data protection provider via DI.
- (c) Creating a protector.
- (d) Protecting then Unprotecting data.

- for example,

using System;

using Microsoft.AspNetCore.DataProtection;

using Microsoft.Extensions.DependencyInjection;

public class Program {

```
public static void Main(string[] args) {
    var serviceCollection = new ServiceCollection();
    serviceCollection.AddDataProtection();
    var services = serviceCollection.BuildServiceProvider();
    var instance = ActivatorUtilities.CreateInstance
        <MyClass>(services);
    instance.RunSample();
}
```

```
public class MyClass
{
    IDataProtector _protector;
    public MyClass(IDataProtectionProvider provider) {
        _protector = provider.CreateProtector
            ("Contoso.MyClass.v1");
    }
```

```
    public void RunSample() {
        Console.WriteLine("Enter input:");
        string input = Console.ReadLine();
```

```
        string protectedPayload = _protector.Protect
            (input);
```

```
        Console.WriteLine($"Protect returned:
            {protectedPayload}");
```

```
        string unprotectedPayload = _protector.
            Unprotect(protectedPayload);
```

```
        Console.WriteLine($"Unprotect returned:
            {unprotectedPayload}");
    }
```

```
}
```

- Instances of "IDataProtectionProvider" and "IDataProtector" are thread-safe for multiple callers. It's intended that once a component gets a reference to an "IDataProtectionProvider" via a call to "CreateProtector", it will use that reference for multiple calls to "protect" and "unprotect".
- A call to "Unprotect" will throw `CryptographicException` if the protected payload can't be verified or deciphered. Some components may wish to ignore errors during unprotect operations; a component which reads authorize and authentication cookies might handle this error and treat the requests as if it had no cookie at all rather than fail the requests outright. A component which wants this behavior should specifically catch `CryptographicException` instead of a swallowing all exceptions.

Interfaces:-

- (1) `IDataProtectionProvider`: The provider interface represents the root of the data protection system. It cannot directly be used to protect or unprotect data. Instead, the consumer must get a reference to an `IDataProtector` by calling `IDataProtectionProvider.CreateProtector(purpose)`, where `purpose` is a string that describes the intended consumer use case. See Purpose Strings for much more information on the intended intent of this parameter and how to choose an appropriate value.
- (2) `IDataProtector`: The protector interface is returned by a call to `CreateProtector`, and it's this interface which consumers can use to perform protect and unprotect

Operations.

- To protect a piece of data, pass the data to the `Protect` method. The basic interface defines a method which converts `byte[]` $\rightarrow$ `byte[]`, but there's also an Overload (provided by as an extensions method) which converts `String` $\rightarrow$ `String`.
- The security offered by the two methods is identical; the developer should choose whichever Overload is most convenient for their use case. Irrespective of the Overload chosen, the value returned by the `Protect` method is now protected (enciphered and tamperproofed), and the application can send it to an untrusted client.

Globalization and localization in ASP.NET Core :

- (1) Globalization (G11N): The process of making an app support different languages and regions. The abbreviation comes from the first and last letters and the number of letters between them.
- (2) Localization (L10N): The process of customizing a globalized app for specific languages and regions.
- (3) Internationalization (I18N): Both globalization and localization.
- (4) Culture: A language and, optionally, a regions.
- (5) Neutral culture: A culture that has a specified language, but not a regions (for example, "en", "es").
- (6) Specific culture: A culture that contains has a specified language and region (for example, "en-US", "en-GB", "en-CL").
- (7) Parent culture: The neutral culture that contains a specified culture (for example, "en" is the parent culture of "en-US" and "en-GB").

(8) Locale : A locale is the same as a culture.

Migrate from ASP.NET Framework to ASP.NET Core :

- Migration tools and resources :

a) Automated assistance :

- .NET Upgrade Assistant - Command-Line tool for initial project conversion.
- Visual Studio Extension - GUI-based upgrade assistance.

b) Comprehensive guides :

- Project Porting ASP.NET Apps eBook - Complete references guide.
- eShop Migration Example - Real-World case study.